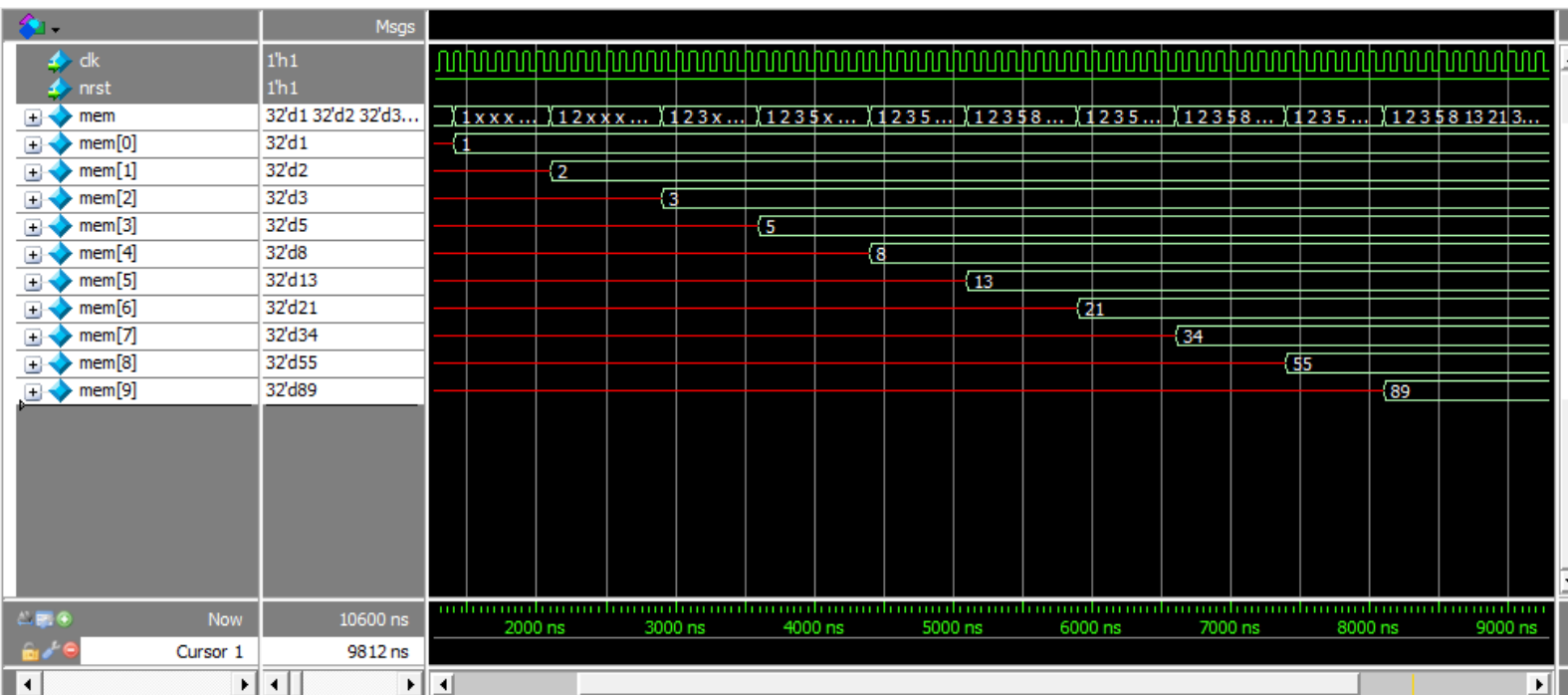






```
1  module ALU(  
2  input signed  [31 : 0] SrcA, SrcB,  
3  input          [2 : 0] ALUControl,  
4  
5  output zero, signflag,  
6  output reg signed [31 : 0] ALUResult  
7  );  
8  
9  always@(*) begin  
10     case(ALUControl)  
11         0: ALUResult = SrcA + SrcB;  
12         1: ALUResult = SrcA << SrcB;  
13         2: ALUResult = SrcA - SrcB;  
14         3: ALUResult = SrcA ^ SrcB;  
15         4: ALUResult = SrcA >> SrcB;  
16         5: ALUResult = SrcA | SrcB;  
17         6: ALUResult = SrcA & SrcB;  
18         default : ALUResult = 0;  
19     endcase  
20 end  
21 assign zero = (ALUResult == 0);  
22 assign signflag = ALUResult[31];  
23 endmodule
```

```

1  module controlUnit (
2      input  [6:0] opcode,
3      input  [2:0] funct3,
4      input      funct7,
5      input      Zero,
6      input      Sign,
7      output     PCSrc,
8      output     ResultSrc,
9      output     MemWrite,
10     output     ALUSrc,
11     output     RegWrite,
12     output [1:0] ImmSrc,
13     output reg [2:0] ALUControl
14 );
15
16     wire [1:0] ALUOp;
17
18     //*****
19     // Main Decoder
20     //*****
21     assign RegWrite  = (opcode == 7'b0000011 || opcode == 7'b0010011 || opcode == 7'b0110011);
22     assign ImmSrc[0] = (opcode == 7'b0100011);
23     assign ImmSrc[1] = (opcode == 7'b1100011);
24     assign ALUSrc    = (opcode == 7'b0000011 || opcode == 7'b0100011 || opcode == 7'b0010011);
25     assign MemWrite  = (opcode == 7'b0100011);
26     assign ResultSrc = (opcode == 7'b0000011);
27     assign PCSrc     = (opcode == 7'b1100011) & (funct3 == 3'b000 ? Zero : funct3 == 3'b001 ? ~Zero : Sign );
28
29     // ALUOp assignments
30     assign ALUOp[0] = (opcode == 7'b1100011);
31     assign ALUOp[1] = (opcode == 7'b0110011 || opcode == 7'b0010011);
32
33     //*****
34     // ALU Decoder
35     //*****
36     always @(ALUOp or funct3 or {opcode[5], funct7}) begin
37         case(ALUOp)
38             2'b00 : begin
39                 ALUControl = 3'b000;
40             end
41
42             2'b01 : begin
43                 if(funct3 == 3'b000 || funct3 == 3'b001 || funct3 == 3'b100) begin
44                     ALUControl = 3'b010;
45                 end
46             end
47
48             2'b10 : begin
49                 if(funct3 == 3'b000 && {opcode[5], funct7} == 2'b11) begin
50                     ALUControl = 3'b010;
51                 end
52                 else begin
53                     ALUControl = funct3;
54                 end
55             end
56
57             default : begin
58                 ALUControl = 3'b000;
59             end
60         endcase
61     end
62
63 endmodule

```



```
1  module DataMemo(  
2      input clk,WE,  
3      input [31 : 0] WD , A,  
4      output [31 : 0] RD  
5  );  
6  
7  reg [31 : 0] mem [0 : 63];  
8  
9  always @(posedge clk ) if(WE) mem[A[31:2]] <= WD;  
10  
11  assign RD = mem[A[31:2]];  
12  endmodule  
13
```



```
1  module instructionMemory(  
2  
3      input [31 : 0] pc,  
4      output [31 : 0] instr  
5  );  
6  reg [31 : 0] mem [0 : 63];  
7  initial $readmemh("program.txt", mem);  
8  
9  assign instr = mem[pc[31 : 2]];  
10  
11  
12  
13  
14  endmodule
```



```
1  module ProgramCounter(  
2  
3  input clk , nrst, PCSrc ,load,  
4  input [31 : 0] immExt,  
5  output reg [31 : 0] PC  
6  
7  );  
8  wire [31 : 0] PCNext;  
9  
10 always@(posedge clk or negedge nrst) begin  
11 if(!nrst) PC <= 0;  
12 else if(load) PC <= PCNext ;  
13 end  
14  
15 assign PCNext = PCSrc ? PC + immExt : PC + 4 ;  
16  
17 endmodule
```



```
1  module RegFile(  
2  input clk, nrst, WE3,  
3  input [4 : 0] A1, A2, A3,  
4  input [31 : 0] WD3,  
5  output [31 : 0] RD1, RD2  
6  );  
7  integer i;  
8  reg [31 : 0] mem[0 : 31];  
9  
10 always@(posedge clk or negedge nrst) begin  
11  
12 if(!nrst)    for( i = 0; i< 32 ; i= i+1)mem[i] <= 0 ;  
13 else if(WE3)    mem[A3] <= WD3;  
14  
15 end  
16  
17 assign RD1 = nrst ? mem[A1] : 0;  
18 assign RD2 = nrst ? mem[A2] : 0;  
19  
20 endmodule
```




```
1  module SignExtend(  
2      input [24:0] Instr,  
3      input [1:0] ImmSrc,  
4      output reg [31:0] ImmExt  
5  );  
6  
7  always @(*) begin  
8      case(ImmSrc)  
9          2'b00: // I-type  
10             ImmExt = {{20{Instr[24]}}, Instr[24:13]};  
11          2'b01: // S-type  
12             ImmExt = {{20{Instr[24]}}, Instr[24:18], Instr[4:0]};  
13          2'b10: // B-type  
14             ImmExt = {{20{Instr[24]}}, Instr[0], Instr[23:18], Instr[4:1], 1'b0};  
15          default:  
16             ImmExt = 32'b0;  
17      endcase  
18  end  
19  
20  endmodule
```



```

1  module RISC_V(
2      input clk, nrst
3  );
4
5      wire [31:0] wire_PC;
6      wire [31:0] wire_ImmExt;
7
8      wire [31:0] wire_instr;
9      wire wire_PCSrc;
10     wire wire_ResultSrc;
11     wire wire_MemWrite;
12     wire wire_ALUSrc;
13     wire wire_RegWrite;
14     wire [1:0] wire_ImmSrc;
15     wire [2:0] wire_ALUControl;
16     wire wire_zero;
17     wire [31:0] wire_RD1;
18     wire [31:0] wire_RD2;
19     wire [31:0] wire_out;
20     wire wire_signflag;
21     wire signed [31:0] wire_ALUResult;
22     wire [31:0] wire_RD;
23
24     // Program Counter
25     ProgramCounter pc_inst (
26         .clk(clk),
27         .nrst(nrst),
28         .PCSrc(wire_PCSrc),
29         .load(1'b1),
30         .immExt(wire_ImmExt),
31         .PC(wire_PC)
32     );
33
34     // Instruction Memory
35     InstructionMemory imem (
36         .pc(wire_PC),
37         .instr(wire_instr)
38     );
39
40     // Control Unit
41     controlUnit controlUnit_inst (
42         .opcode(wire_instr[6:0]),
43         .funct3(wire_instr[14:12]),
44         .funct7(wire_instr[30]),
45         .Zero(wire_zero),
46         .Sign(wire_signflag),
47         .PCSrc(wire_PCSrc),
48         .ResultSrc(wire_ResultSrc),
49         .MemWrite(wire_MemWrite),
50         .ALUSrc(wire_ALUSrc),
51         .RegWrite(wire_RegWrite),
52         .ImmSrc(wire_ImmSrc),
53         .ALUControl(wire_ALUControl)
54     );
55
56     // Register File
57     RegFile RegFile_inst (
58         .clk(clk),
59         .nrst(nrst),
60         .WE3(wire_RegWrite),
61         .A1(wire_instr[19:15]),
62         .A2(wire_instr[24:20]),
63         .A3(wire_instr[11:7]),
64         .WD3(wire_out),
65         .RD1(wire_RD1),
66         .RD2(wire_RD2)
67     );
68
69     // ALU
70     wire [31:0] alu_src_b = wire_ALUSrc ? wire_ImmExt : wire_RD2;
71     ALU ALU_inst (
72         .SrcA(wire_RD1),
73         .SrcB(alu_src_b),
74         .ALUControl(wire_ALUControl),
75         .zero(wire_zero),
76         .signflag(wire_signflag),
77         .ALUResult(wire_ALUResult)
78     );
79
80     // Data Memory
81     DataMemo DataMemo_inst (
82         .clk(clk),
83         .WE(wire_MemWrite),
84         .WD(wire_RD2),
85         .A(wire_ALUResult),
86         .RD(wire_RD)
87     );
88
89     // Result MUX
90     mux mux_inst (
91         .sel(wire_ResultSrc),
92         .in1(wire_ALUResult),
93         .in2(wire_RD),
94         .out(wire_out)
95     );
96
97     // Sign Extender
98     SignExtend SignExtend_inst (
99         .Instr(wire_instr[31:7]),
100        .ImmSrc(wire_ImmSrc),
101        .ImmExt(wire_ImmExt)
102    );
103    endmodule
104

```